

오너블리

나만의 공간을 나만의 분위기로 표현하다

팀
팀장
팀원

버그 샌드위치
정**
김** 변** 허완

목차

1. 개요
2. 요구사항 분석
ERD
3. User Flow
Logic Process
4. 기능 소개
에러 발생 원인 및 해결방안
5. 프로젝트 회고

팀 소개



버그 샌드위치

개발 과정에서 마주치는 다양한 버그를
하나씩 개발 일지에 끼워 넣어
원인을 분석하고 이를 해결해 나가는 팀



팀원 소개



정**

- Spring Security
- SNS 로그인 통합
- 회원 관련 서비스



김**

- DB 이관 작업
- 이벤트 로직
- 백엔드



변**

- 백엔드
- 결제 로직
- 이메일 API



허 완

- 구조 설계
- 프론트
- 병합 작업

프로젝트 개요



프로젝트 명 : 오너블리

기분과 감정적 만족을 위한 소비를 일컫는 단어, '필코노미'
2025 하반기 부터 트렌드 확산
이러한 흐름에 잘 맞는 오너먼트를 주제로 선정



프로젝트 목적

REST API와 다양한 프레임워크를 활용
역할 분배를 포함한 개발 과정을 경험



프로젝트 기간

총 42일
2026.1.15 ~ 2026.02.25

프로젝트 기간 : 총 42일

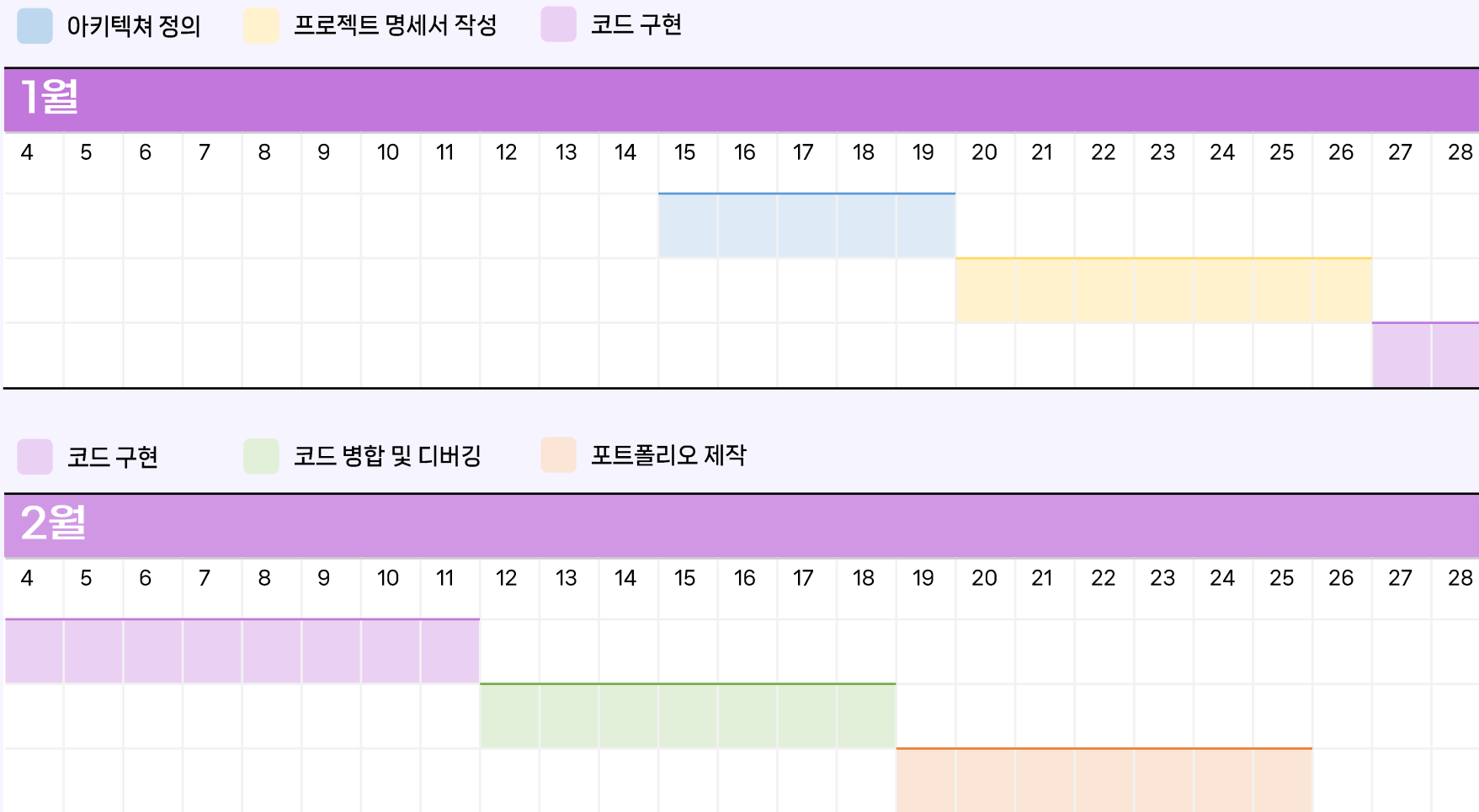
프로젝트 목표설정
2026-01-15

프로젝트 설계 시작
2026-01-20

프로젝트 구현 시작
2026-01-27

프로젝트 병합 및 디버깅
2026-02-12

↓ 프로젝트 완성
2026-02-19

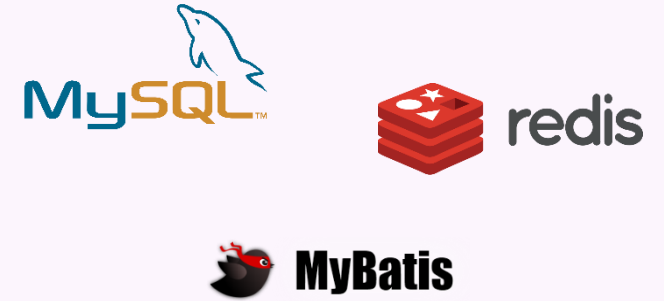


개발 환경

백엔드



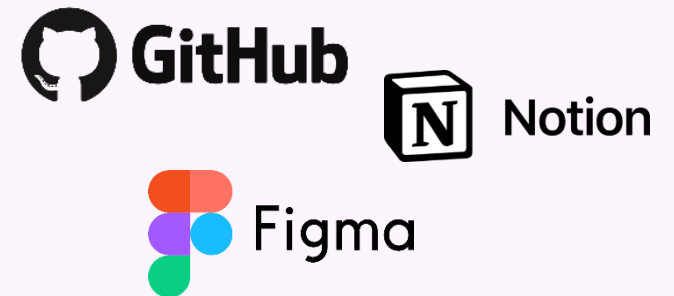
DB



프론트



협업



요구사항 분석



이벤트 추가



광고 이메일



회원 관점의
페이지



전자상거래법
고려



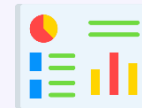
2개 이상의
결제 방식



반응형 웹



트랜잭션 처리



관리자 전용
대시보드

ERD

회원			
PK	ACCOUNT_PK	INTEGER	회원 고유 번호
	ACCOUNT_ID	VARCHAR(50)	회원 아이디
	ACCOUNT_PASSWORD_HASH	VARCHAR(100)	회원 비밀번호 해시 값
	ACCOUNT_NAME	VARCHAR(50)	회원 이름
	ACCOUNT_EMAIL	VARCHAR(100)	회원 이메일
	ACCOUNT_PHONE	VARCHAR(11)	회원 전화번호
	ACCOUNT_DATE	DATETIME	회원 가입일
	ACCOUNT_ROLE	VARCHAR(50)	회원 권한
	ACCOUNT_EVENT_OPT_IN	TINYINT(1)	이벤트 수신 동의 여부

주소			
PK	ADDRESS_PK	INTEGER	주소 고유 번호
FK	ACCOUNT_PK	INTEGER	회원 FK
	ADDRESS_NAME	VARCHAR(50)	배송지명
	ADDRESS_IS_DEFAULT	TINYINT(1)	기본 배송지 여부
	ADDRESS_POSTAL_CODE	VARCHAR(20)	우편번호
	ADDRESS_REGION	VARCHAR(100)	기본 주소
	ADDRESS_DETAIL	VARCHAR(200)	상세 주소

상품			
PK	ITEM_PK	INTEGER	상품 고유 번호
	ITEM_NAME	VARCHAR(100)	상품명
	ITEM_PRICE	INTEGER	상품 가격
	ITEM_STOCK	INTEGER	상품 재고
	ITEM_DESCRIPTION	VARCHAR(3000)	상품 설명
	ITEM_IMAGE_URL	VARCHAR(255)	상품 이미지 경로
	ITEM_CATEGORY	VARCHAR(200)	상품 카테고리
	ITEM_REGIST_DATE	DATETIME	상품 등록일

장바구니			
PK	CART_PK	INTEGER	장바구니 고유 번호
FK	ACCOUNT_PK	INT	회원 FK
FK	ITEM_PK	INTEGER	상품 FK
	CART_COUNT	INTEGER	장바구니 수량

접속로그			
PK	CONNECT_LOG_PK	INTEGER	접속 로그 고유 번호
FK	ACCOUNT_PK	INTEGER	회원 FK
	CONNECT_IP	VARCHAR(50)	접속 IP
	CONNECT_DEVICE	VARCHAR(200)	접속 환경(기기/브라우저)
	CONNECT_DATE	DATETIME	접속 날짜/시간

찜			
PK	WISHLIST_PK	INTEGER	찜 고유 번호
FK	ACCOUNT_PK	INTEGER	회원 FK
FK	ITEM_PK	INTEGER	상품 FK

주문			
PK	ORDERS_PK	INTEGER	주문 고유 번호
FK	ACCOUNT_PK	INTEGER	회원 FK
	ORDERS_DATE	DATETIME	주문 날짜
	ORDERS_STATUS	VARCHAR(30)	주문 상태
	ADDRESS_NAME	VARCHAR(50)	배송지 명
	ORDERS_PAYMENT_TYPE	VARCHAR(50)	결제 방식
	ORDERS_IMPORT_UID	VARCHAR(100)	결제 고유 ID
	ORDERS_MESSAGE	VARCHAR(500)	주문 요청 사항

후기			
PK	REVIEW_PK	INTEGER	후기 고유 번호
FK	ACCOUNT_PK	INTEGER	회원 FK
FK	ITEM_PK	INTEGER	상품 FK
	REVIEW_TITLE	VARCHAR(50)	후기 제목
	REVIEW_CONTENT	VARCHAR(4000)	후기 내용
	REVIEW_DATE	DATETIME	후기 작성일
	REVIEW_STAR	TINYINT(2)	후기 평점
	REVIEW_IMAGE_URL	VARCHAR(255)	후기 이미지 경로

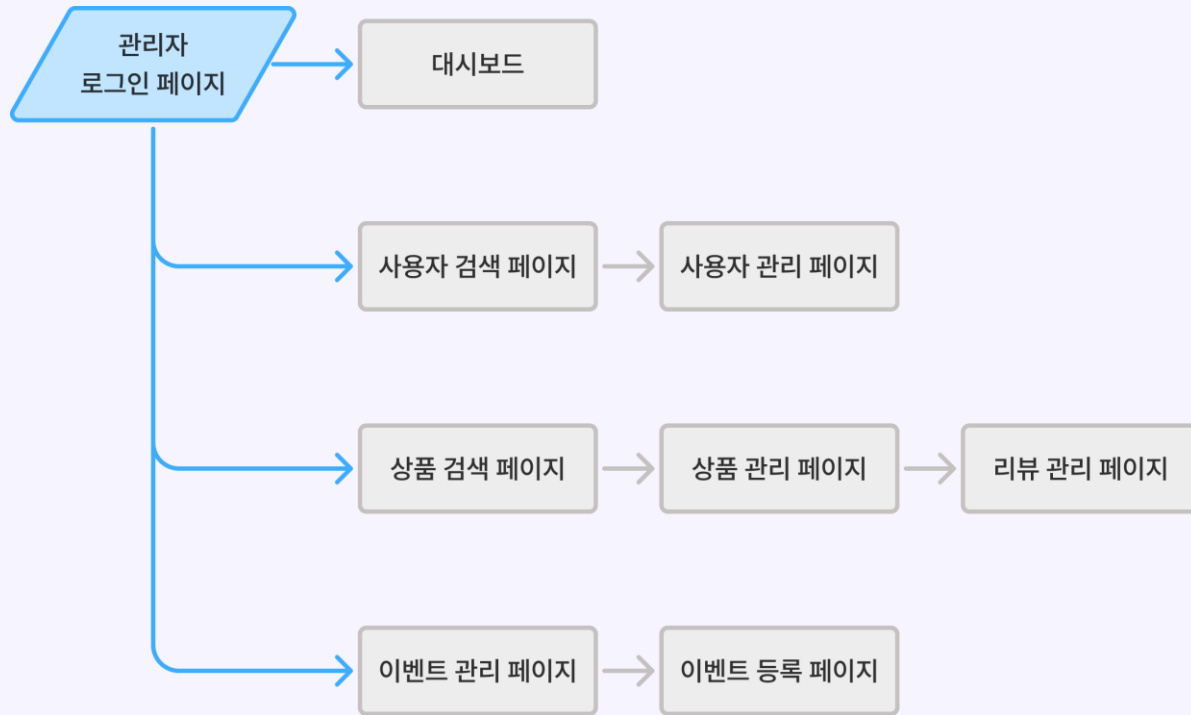
주문상세			
PK	ORDERS_ITEM_PK	INTEGER	주문 상세 고유 번호
FK	ORDERS_PK	INTEGER	주문 FK
FK	ITEM_PK	INTEGER	상품 FK
	ORDERS_ITEM_COUNT	INTEGER	주문 수량
	ORDERS_ITEM_PRICE	INTEGER	주문 시점 가격

이벤트			
PK	EVENT_PK	INTEGER	이벤트 고유 번호
	EVENT_NAME	VARCHAR(200)	이벤트 이름
	EVENT_START_DATE	DATETIME	이벤트 시작일
	EVENT_END_DATE	DATETIME	이벤트 종료일
	EVENT_TARGET_ACCOUNT	JSON	이벤트 대상
	EVENT_TARGET_CATEGORY	JSON	이벤트 상품 카테고리
	EVENT_DESCRIPTION	VARCHAR(3000)	이벤트 설명
	EVENT_DISCOUNT_RATE	INTEGER	할인율
	EVENT_IMAGE_URL	VARCHAR(255)	이벤트 이미지 경로

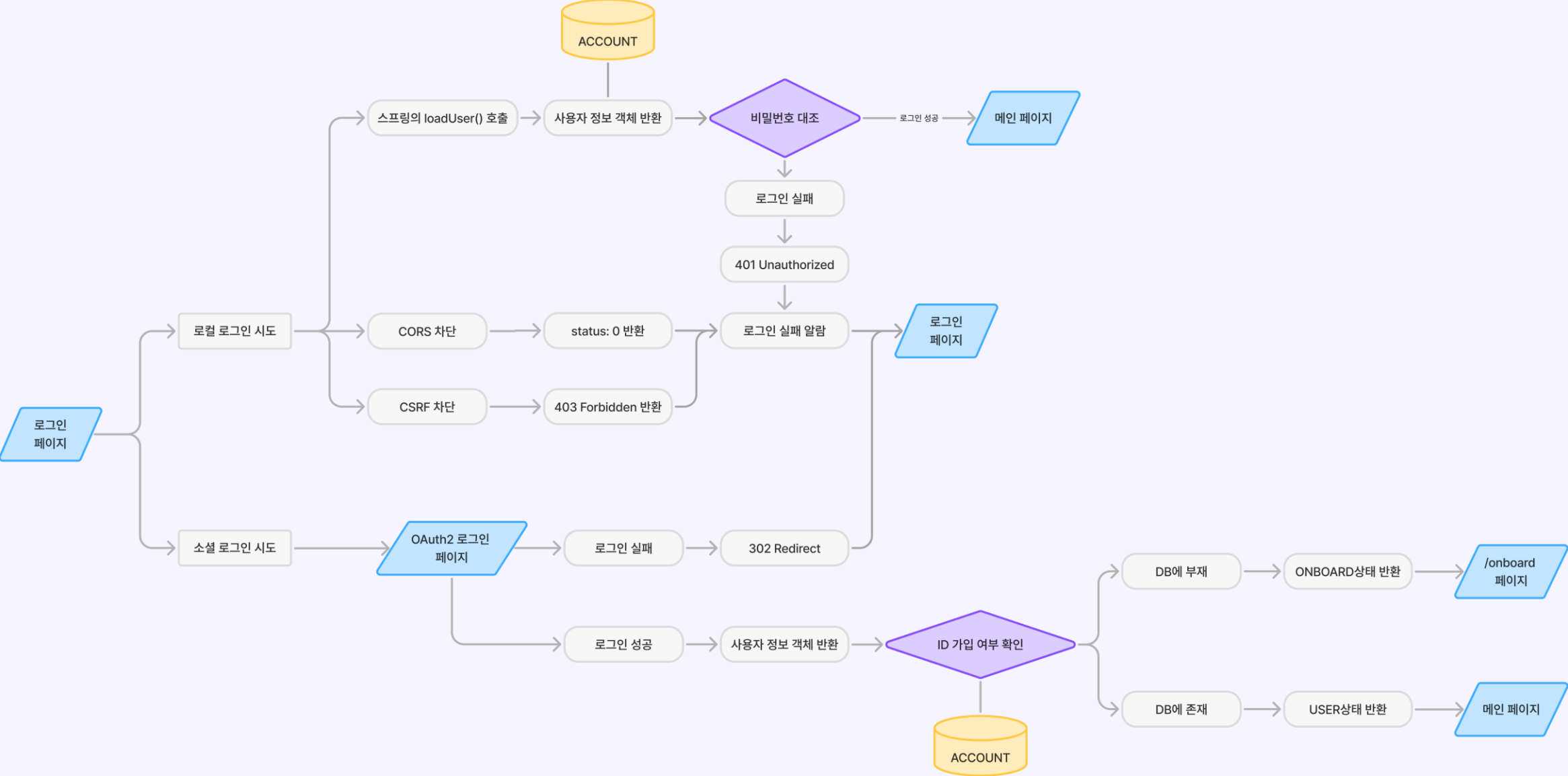
USER FLOW - 사용자



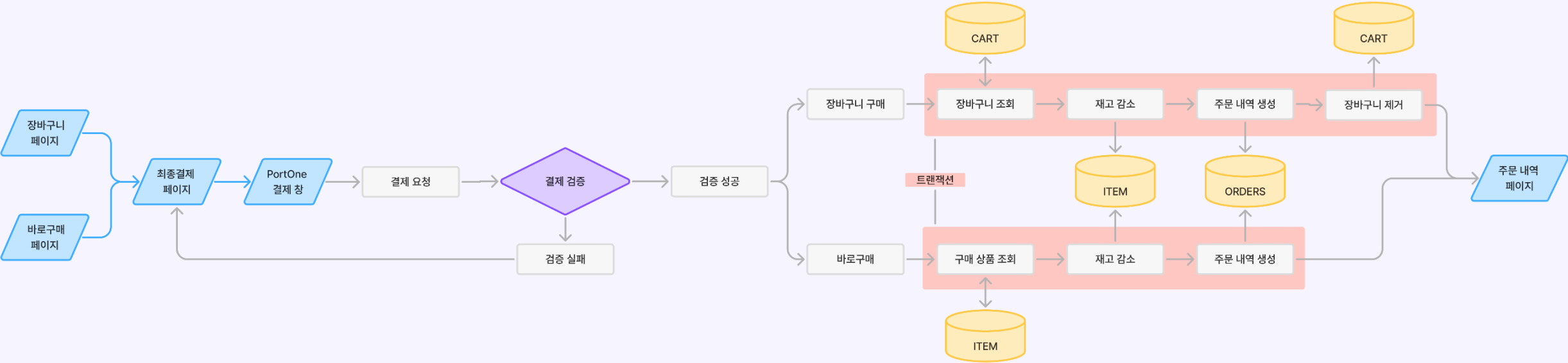
USER FLOW - 관리자



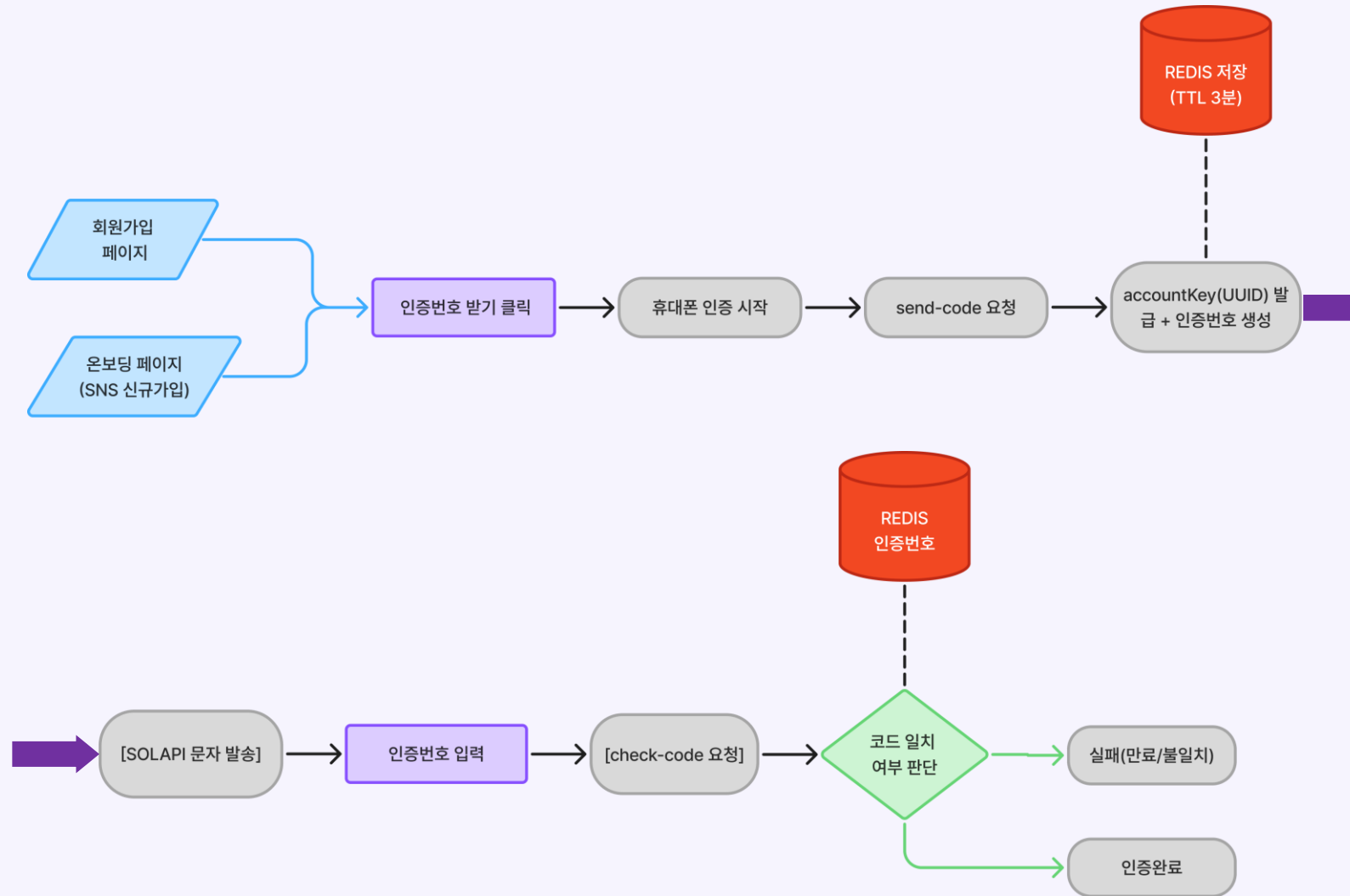
LOGIC PROCESS - 로그인



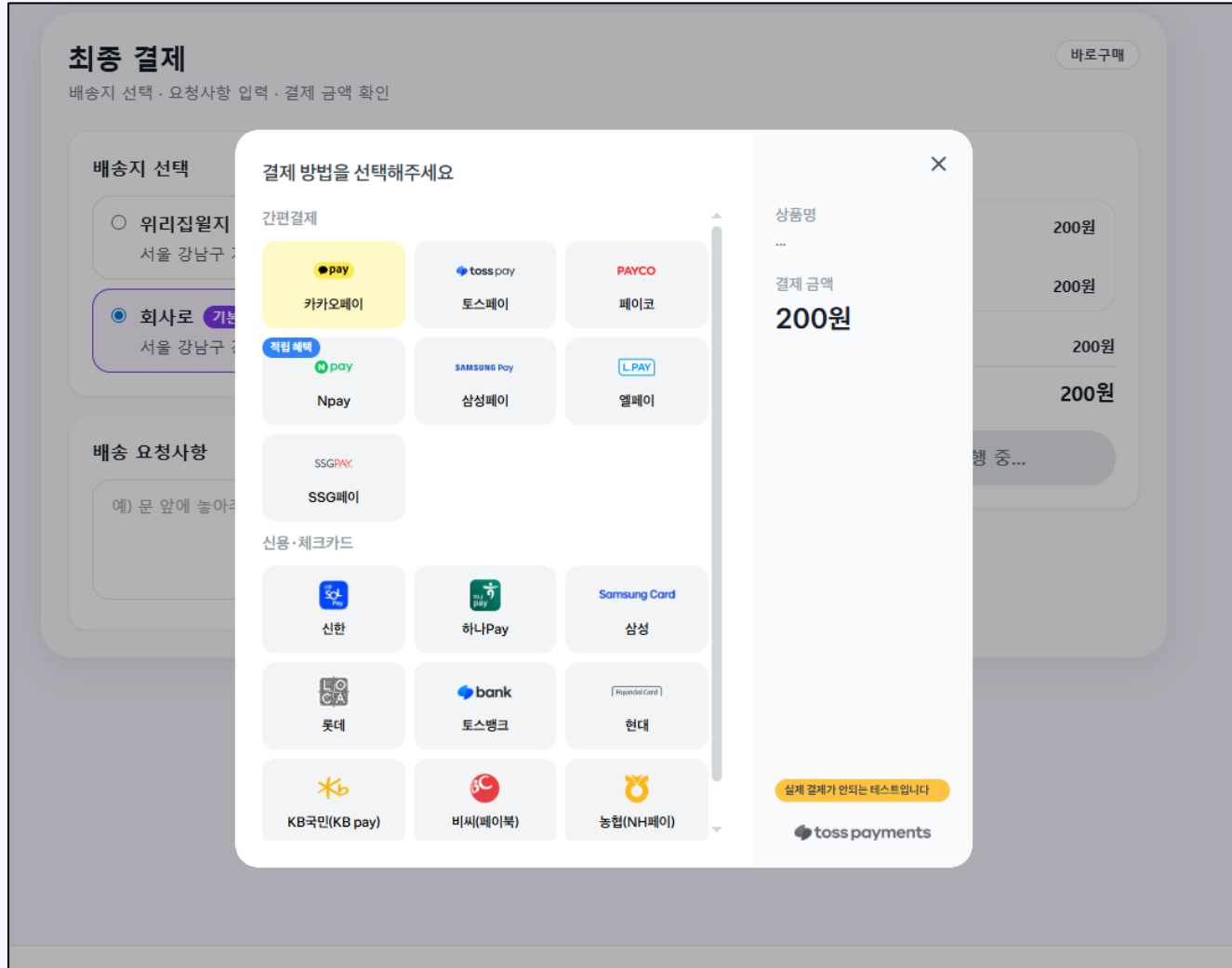
LOGIC PROCESS - 결제



LOGIC PROCESS - 전화번호 검증



UI / UX 개선 - 결제수단 추가



간편 결제부터 카드 결제까지
다양한 결제수단 이용 가능

UI / UX 개선 – SNS 로그인 방식 추가

kakao

카카오메일 아이디, 이메일, 전화번호

비밀번호

☐ 간편로그인 정보 저장 ⓘ

로그인

또는

QR코드 로그인

[회원가입](#) [계정 찾기](#) | [비밀번호 찾기](#)

한국어 ▼ | [이용약관](#) | [개인정보 처리방침](#) | [고객센터](#) | © Kakao Corp.

NAVER

ornably 이용을 위해 네이버에 로그인해 주세요.

⚠ 여러 사람이 쓰는 PC라면
① '로그인 상태 유지' 체크하지 않기 ② 사용 후 로그아웃하기

ID/전화번호

[1] 일회용 번호

QR코드

아이디 또는 전화번호

비밀번호

☒ 로그인 상태 유지 IP보안 ON

로그인

[비밀번호 찾기](#) | [아이디 찾기](#) | [회원가입](#)

Google 계정으로 로그인

로그인

이메일 또는 휴대전화

pring OAuth Test(으)로 이동

이메일을 잊으셨나요?

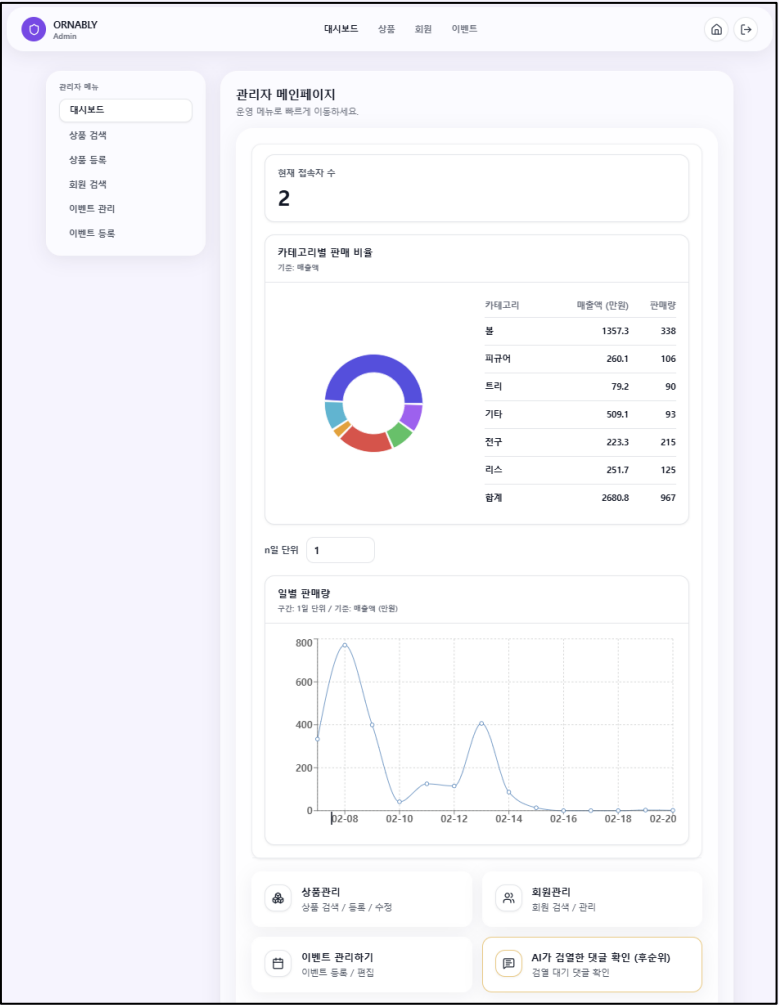
[계정 만들기](#) [다음](#)

한국어 ▼

[도움말](#) [개인정보처리방침](#) [약관](#)

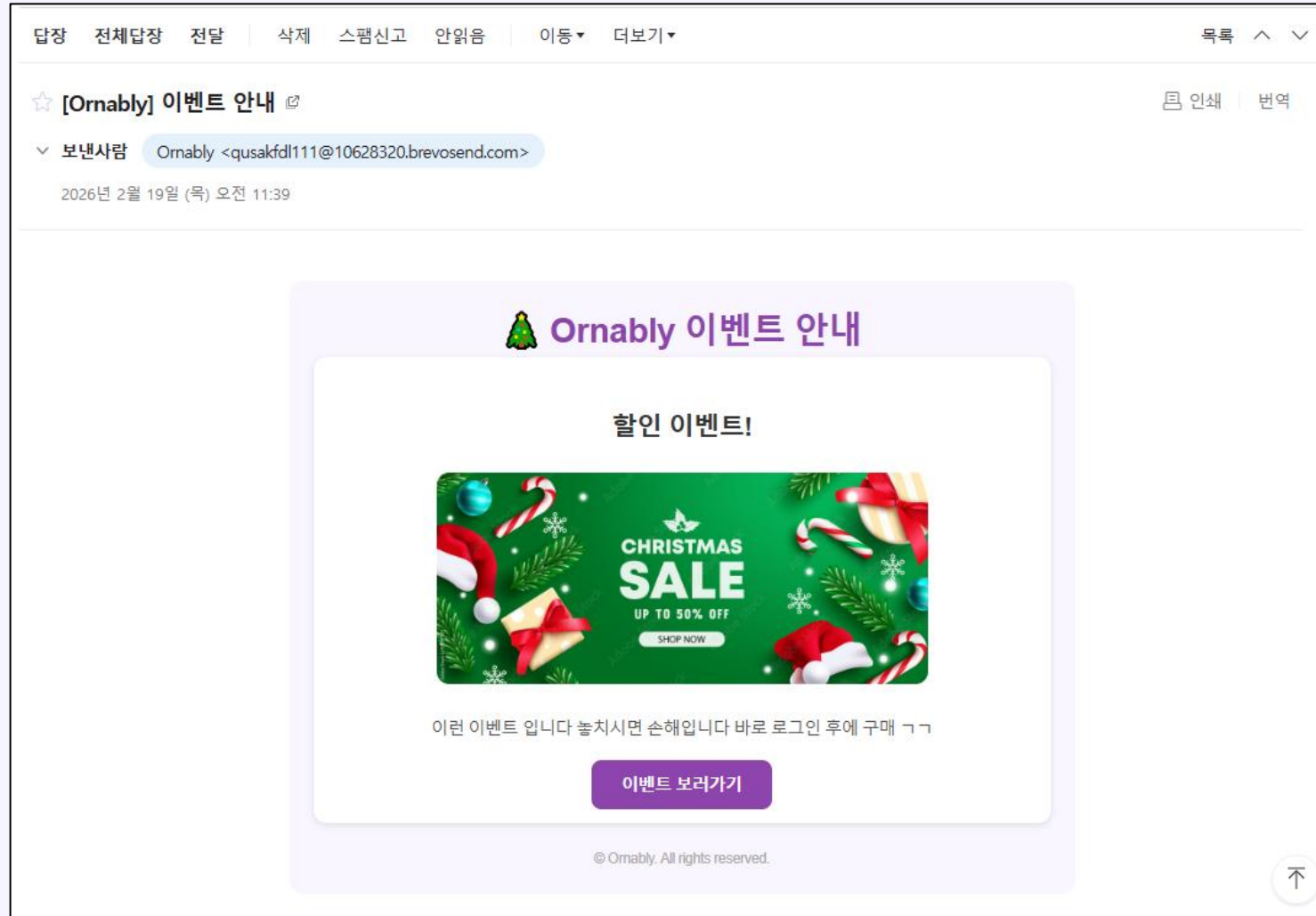
기존 : 회원 로그인, 카카오
변경 : 회원 로그인, 카카오, 네이버, 구글

기능 추가 – 관리자 기능 추가



관리자 페이지에서 로그인 후
상품 통계 및 회원 관리 기능 추가

UI / UX 개선 – 광고 이메일 제공



관리자가 이벤트 등록 시
이벤트 수신 동의한 회원에게
광고 이메일 발송

기능 추가 – 이벤트 기능 추가

시즌 이벤트

최대 할인 이벤트를 확인해보세요

★ 최대 30%

2026-02-14 ~ 2026-03-02

설레는 설날엔 30% 할인 이벤트

새해를 맞아 준비한 특별한 혜택!
설 연휴 동안 전 상품 최대 30% 할인 이벤트를 진행합니다.

★ 이벤트 기간

2026년 2월 14일(토) ~ 2026년 3월 2일(월)

이벤트 혜택

전 피규어 상품 최대 30% 즉시 할인



현재 진행중인 이벤트 확인 가능

기능 추가 – 비밀번호 암호화

ACCOUNT_PK	ACCOUNT_ID	ACCOUNT_PASSWORD_HASH	ACCOUNT_NAME	ACCOUNT_EMAIL
14	heejin	\$2a\$12\$3o84gJLNbZ5kGUPwRrBUd.RaTaXrda.0bBJDwy6NALNf0Jp8ua8ey	변희인	quasakfe
16	yoojin0819	\$2a\$12\$vV4CBISeA9youp.IgflsyuPzuUoDz31VaUCisugxSr6fHYjEqFQe2	김유진	yoojin0
17	seohwa56	\$2a\$12\$kYSoBKZeGxe1Yg2vRkaNc.aEZmR1h/vc.jG6iKBf.XJoE1JODRYo6	정송이	seohwa5
18	hurwan	\$2a\$12\$ZUNQisQFMK8sJXP6KKi6AeKJ5em9rHpCA9QLWfbCj.PQtLScnvRZy	허완	hurwan0
19	hurwan0629	\$2a\$12\$HrIZ1wWv/FcUqYIW3GCGG.sIvTTPMzodPj4oRfQK4jHqKG.0TPq4e	허완	hurwan0
20	hong1234	\$2a\$12\$0HNt.BaOILb9FBJlp7eAPuT.AjFpickt3.bUyQ81KISSbzJaqkZ3m	홍길동	hong123
21	jihoon_kim	\$2a\$12\$x8N5jh83VARG6AEj05068.NtNOSrm5W3CiS.QOU.P0tUrXsP00oa6	김지훈	jihoon_k
22	seoyeon_lee	\$2a\$12\$4F8NeCfC5PVH2eLVLSzS80173WmbBfbXlqg7cpLAKLdVHVnnLFDdK	이서연	seoyeon
23	minjun_park	\$2a\$12\$vc2l0nr5W1l.y0zcLVLS6eC1WizeD8ZDm.Kf6mnaT54FYth06pspm	박민준	minjun_p
24	NULL	\$2a\$12\$7yR7887Che0d5xoTnpMjYuvZeaLvCY3uQo2ho2w17tI9hBsX6B2Vu	최유나	yuna_ch
25	dohyun_jung	\$2a\$12\$JQt6dmBZ/1E5hAqCcv.pNuZYOGH8BxNfI03CF4cz50kDaNckkNfdC	정도현	dohyun_j
26	jimin_han	\$2a\$12\$DmgJeJHaEa1jWrNtUrbCEuJyj5SeKcWmxJ0jJqe4NTDjYrgA6sZg.	한지민	jimin_h
27	sehun_oh	\$2a\$12\$IDzRdBGSKCTMRX23pvrXE0miINuf.33.GL1FmgHciZnbt9D0BaHOK	오세훈	sehun_oh
28	yerin_shin	\$2a\$12\$4j/M/u.mkjUmA0dRyEdQUu2La6YGuQL3GDYHzm90po4riqVPFADcm	신예린	yerin_sh
29	taekyung_yoon	\$2a\$12\$/VfCmvSUB8/0NtBQtszXa.0pV3./4QFR5jlqYGcPKAq6wrHci9cHu	윤태경	taekyung
30	haneul_jang	\$2a\$12\$gGcLNxwnV9LA7qeV7eex1.zHKZHIWMIuCGDxpbAyGq.DBSnDA0oZ6	장하늘	haneul_j
31	NULL	NULL	허완	mangins
32	NULL	NULL	허완	hurwan0
33	NULL	NULL	허완	mangins
34	kakao_4675022132	NULL	허완	mangins
35	NULL	NULL	허완	hurwan0
36	NULL	NULL	허완	hurwan0
37	NULL	NULL	허완	hurwan0
38	cchamppang0629	\$2a\$12\$gitwfZhz0YeUwnQ6/WiDJ0rXRHJSJiANBZ3h9aPcMfNAWNeNSkoCu	허완	cchampp
39	NULL	NULL	허완	hurwan0
40	naver_1G1VWEYAuzMo2FLwvC3iyr-mncNo7w84qV37n8RMcnU	NULL	허완	hurwan0
41	google_113732774513265086848	NULL	0in 2	gimzer0
42	kakao_4759371568	NULL	이현빈	lhbwor
43	cchamppang	\$2a\$12\$WyXv.0wTOJqL50AeWC59Qu0SAYBPridiMFRBTfz15p5WdDRuctYZ2	허완	cchampp
44	google_102934966652344799263	NULL	허완	hurwan0

암호화 강도를 조절할 수 있는 bcrypt를 적용해
비밀번호를 암호화 저장

전자상거래법 – 거래 기록은 일정 기관 보관

← 뒤로

회원 관리

새로고침

회원 이름

최유나

회원 PK

24

최유나 메타정보

탈퇴

아이디
(탈퇴한 회원)

총 구매 금액
2,314,490원

가입일
2026-02-13

회원유형
로컬회원

이벤트 수신 동의: 동의

최유나이 쓴 리뷰 목록

총 3개



이건 괜찮네

#36

2026-02-18 06:37:30

별점 3 / 5

그래도 마음에 안드니까 3점주겠습니다.

리뷰 삭제



ㅋㅋ

#35

2026-02-18 06:37:00

별점 1 / 5

이거 저희 집 초등학생 아들이 만든거랑 큰 차이 없는데요?

리뷰 삭제



너무 별로네요

#34

2026-02-18 06:36:22

별점 1 / 5

이거 떨어지니까 망가져 버렸어요 ㅜ
이래도 되는건가요?...

리뷰 삭제

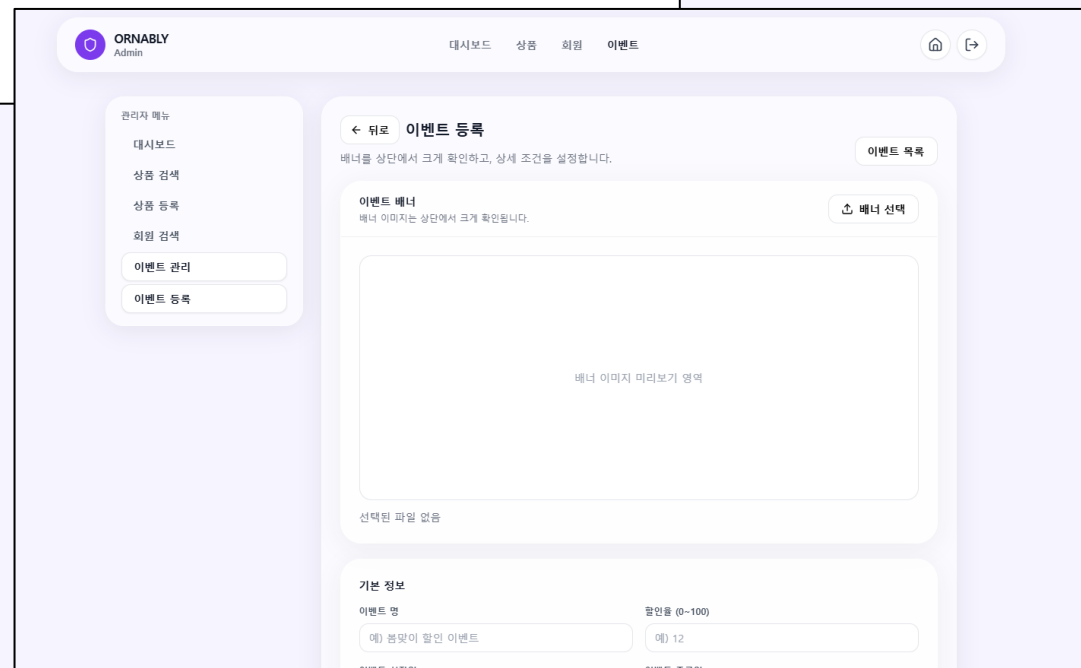
주문 및 결제 기록은 최소 5년 동안 보관해야하기 때문에
회원 탈퇴 시에도 주문 내역은 삭제하지 않고 유지

개인정보 보호를 위해
회원 정보는 삭제 후 회원 상태 변경

MyBatis를 통한 역할 분리

```
EventMapper.xml X
-//mybatis.org//DTD Mapper 3.0//EN (doctype)
1 <?xml version="1.0" encoding="UTF-8" ?>
2 <!DOCTYPE mapper
3   PUBLIC "-//mybatis.org//DTD Mapper 3.0//EN"
4   "http://mybatis.org/dtd/mybatis-3-mapper.dtd">
5
6 <mapper namespace="EventLog">
7
8   <!-- 이벤트 등록 -->
9   <insert id="insert" parameterType="eventDTO">
10     INSERT INTO EVENT
11     (EVENT_NAME, EVENT_IMAGE_URL, EVENT_START_DATE, EVENT_END_DATE,
12     EVENT_TARGET_ACCOUNT, EVENT_TARGET_CATEGORY, EVENT_DISCOUNT_RATE, EVENT_DESCRIPTION)
13     VALUES ({#eventName}, #{eventImageUrl}, #{eventStartDate}, #{eventEndDate}, #{eventTargetAccount}, #{eventTargetCategory}, #{eventDiscountRate}, #{eventDescription})
14   </insert>
15
16   <!-- 이벤트 종료 요청 (종료 날짜 > 종료 요청한 날짜) -->
17   <update id="update" parameterType="eventDTO">
18     UPDATE EVENT
19     SET EVENT_END_DATE = DATE_SUB(NOW(), INTERVAL 1 DAY)
20     WHERE EVENT_PK = #{eventPk}
21   </update>
22
```

- SQL을 XML로 분리하여 응집도를 높이고 결합도 낮춤
- .xml 파일에서 SQL 관리
- MyBatis Namespace 구조로 Repository와 Mapper를 연결



JSON 타입을 통한 복잡한 조건 저장

```
mysql> SELECT EVENT_PK, EVENT_NAME, EVENT_TARGET_ACCOUNT, EVENT_TARGET_CATEGORY, EVENT_DISCOUNT_RATE FROM EVENT;
```

EVENT_PK	EVENT_NAME	EVENT_TARGET_ACCOUNT	EVENT_TARGET_CATEGORY	EVENT_DISCOUNT_RATE
3	신타가 준비한 짝꿍선물	{ "type": "ALL" }	{ "TREE" }	40
4	올레는 올날엔 30% 할인 이벤트	{ "type": "ALL" }	{ "FIGURE" }	30
5	모든 기념 50% 할인 이벤트	{ "type": "ALL" }	{ "LIGHT", "BALL" }	50
6	12	{ "type": "AMOUNT", "amount": 21125 }	{ "WREATHS", "FIGURE" }	12
7	12	{ "type": "ALL" }	{ "WREATHS", "ETC", "FIGURE" }	100
8	올레는 올날엔 30% 할인 이벤트	{ "type": "MEMBER_TYPE", "memberType": ["LOCAL", "GOOGLE", "KAKAO", "NAVER"] }	{ "FIGURE" }	30
9	모든 기념 50% 할인 이벤트	{ "type": "MEMBER_TYPE", "memberType": ["KAKAO", "GOOGLE", "NAVER"] }	{ "TREE", "LIGHT", "FIGURE", "BALL", "WREATHS", "ETC" }	60

7 rows in set (0.00 sec)

이벤트 대상(회원 유형, 카테고리, 구매금액 조건 등)이 자주 변경되는 도메인 특성을 고려
EVENT_TARGET_ACCOUNT,
EVENT_TARGET_CATEGORY
→ JSON 타입으로 설계

이벤트 대상(회원) 설정

대상 유형 선택

모든 사용자

N원 이상 구매 사용자

특정 기간에 회원가입한 사용자

모든 사용자

특정 회원 유형 선택

트러

전구

불

피규어

리스

기타

선택됨: 불, 전구

상세 설정

모든 사용자 대상 (추가 입력 없음)

이벤트 설명

배너 아래에 노출될 설명 텍스트

이벤트 상세 설명을 입력해주세요.

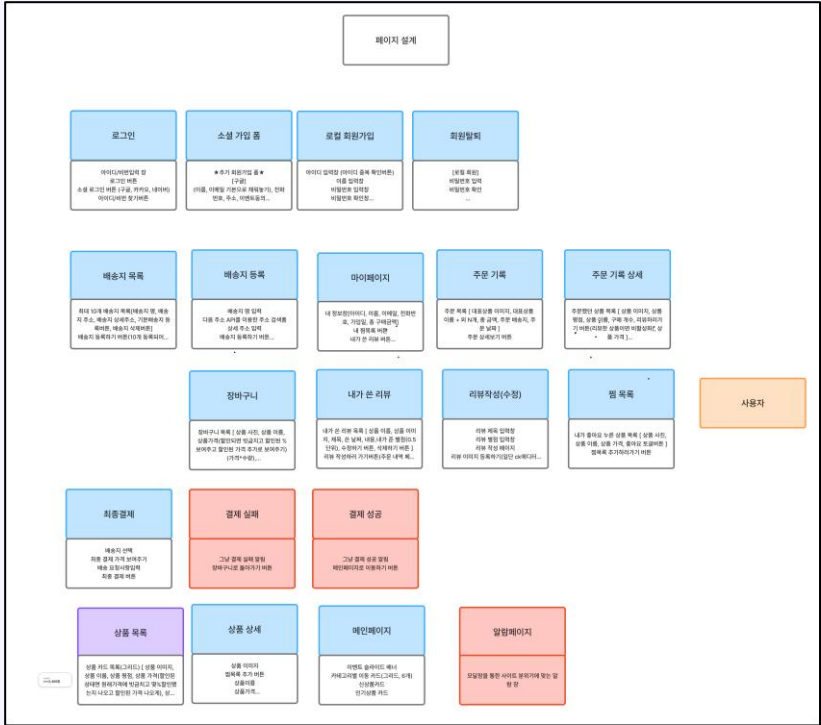
취소

+ 이벤트 등록

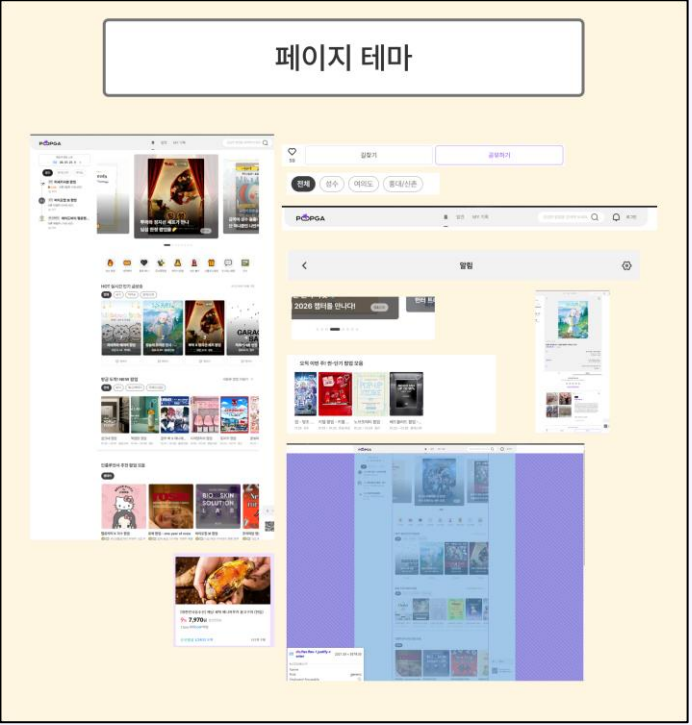
```
// 상품 목록 조회 (이벤트 지원할만큼 + 회원 등록할만큼 + 카테고리/장식/재미등)
private static final String SELECT_ALL_ITEM =
    "SELECT "
    " * "
    " FROM account a "
    " LEFT JOIN orders o ON o.account_pk = a.account_pk "
    " LEFT JOIN orders_item oi ON oi.orders_pk = o.orders_pk "
    " WHERE a.account_role = ? "
    " GROUP BY a.account_pk, date(a.account_date), a.account_role "
    " ) "
    " event_max AS ( "
    " SELECT "
    " * "
    " FROM item i "
    " JOIN event e "
    " ON JSON_CONTAINS(e.event_target_category, JSON_QUOTE(i.item_category)) "
    " AND CURRENT_DATE BETWEEN e.event_start_date AND e.event_end_date "
    " LEFT JOIN acct a ON 1 = 1 "
    " WHERE ( "
    " (a.account_pk IS NULL AND (e.event_target_account->>$.type) = 'ALL' ) "
    " OR "
    " (a.account_pk IS NOT NULL AND ( "
    " (e.event_target_account->>$.type) = 'ALL' "
    " OR "
    " (e.event_target_account->>$.type) = 'AMOUNT' "
    " AND a.totalAmount >= CAST(e.event_target_account->>$.amount AS UNSIGNED) "
    " ) "
    " OR "
    " (e.event_target_account->>$.type) = 'JOINED' "
    " AND a.joinedDate BETWEEN "
    " STR_TO_DATE(e.event_target_account->>$.startDate, '%Y-%m-%d') "
    " AND STR_TO_DATE(e.event_target_account->>$.endDate, '%Y-%m-%d') "
    " ) "
    " OR ( "
    " (e.event_target_account->>$.type) = 'MEMBER_TYPE' "
    " AND JSON_CONTAINS( "
    " JSON_EXTRACT(e.event_target_account, '$.memberType'), "
    " JSON_QUOTE(a.accountRole) "
    " ) "
    " ) "
    " ) "
    " ) "
    " GROUP BY i.item_pk "
    " ) "
    " review_avg AS ( "
    " SELECT "
    " * "
    " FROM review r "
    " JOIN item i ON i.item_pk = r.item_pk "
    " JOIN event e ON e.item_pk = i.item_pk "
    " LEFT JOIN review_avg ra ON ra.item_pk = i.item_pk "
    " WHERE "
    " ( ? = 'ALL' OR i.item_category = ? ) "
    " AND ( ? IS NULL OR ? = '' OR i.item_name LIKE CONCAT('%', ?, '%') ) "
    " ORDER BY "
    " CASE WHEN ? = 'popular' THEN IFNULL(ra.itemAvgStar, 0) END DESC, "
    " CASE WHEN ? = 'discount' THEN IFNULL(m.maxDiscountRate, 0) END DESC, "
    " CASE WHEN ? = 'new-reverse' THEN i.item_pk END ASC, "
    " CASE WHEN ? = 'default' THEN i.item_pk END DESC, "
    " i.item_pk DESC "
    " LIMIT ? OFFSET ? ;
```

페이지 설계

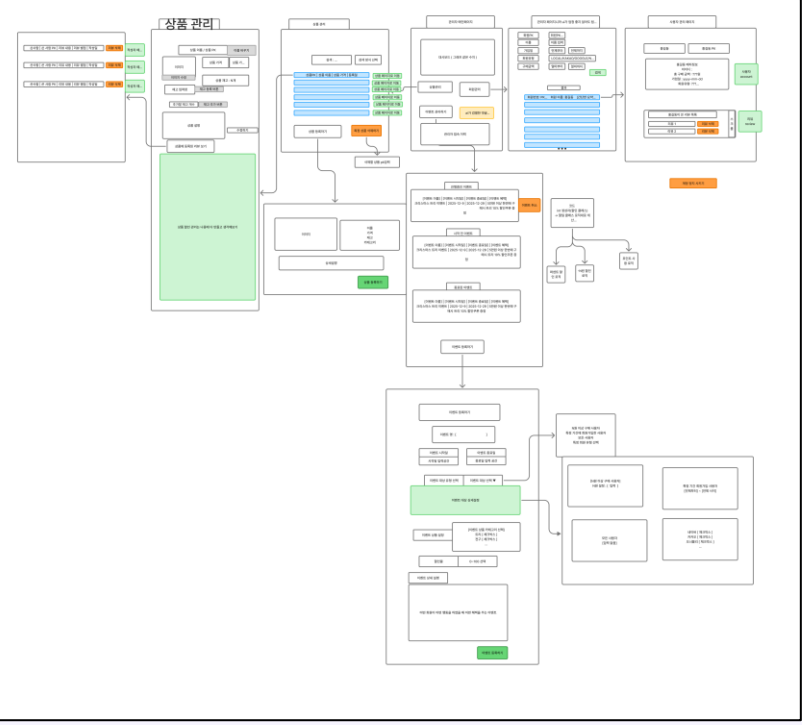
페이지 설계



페이지 분위기



페이지 설명



- 페이지 구성 요소와 디자인 테마를 Figma로 정리하여 서비스 전반의 UI 일관성을 확보
- 진행상황을 시각적으로 관리할 수 있어 일정 조율과 작업 기간 관리에 도움을 줌

API 설계



페이지별 api 문서 정리

웹 url : 페이지 구조
API 읽기 가이드라인
유틸류 API
사용자페이지
비회원 페이지
소셜 회원가입 마무리 (ONBOARD)
회원 페이지
관리자페이지
Forbidden 403

웹 url : 페이지 구조

API 읽기 가이드라인

api url의 형태는 `[http메서드] api/([권한])/([작성 대상 또는 페이지])/([추가 정보])` 로 이루어져 있습니다

- ▶ 사용된 HTTP메서드
- ▶ 권한의 종류
- ▶ auth
- ▶ Content-Type

연관 문서에 가이드라인과 일치하지 않는 url또는 품이 있다면 아래에 작성 바랍니다.

[목록 정보]
-

유틸류 API

 회원 상태 관련 auth*1

사용자페이지

 홈페이지 문서 item*2 event*1
 상품 목록 페이지 item*1
 상품 상세 페이지 wishlist*2 reivew*1 cart*1 item*1

비회원 페이지

 로그인페이지 security*1
 로컬 회원가입 페이지 account*2 solapi*2

소셜 회원가입 마무리 (ONBOARD)

 온보딩 화면 (소셜 회원가입 페이지) account*2 solapi*2

회원 페이지

상품 상세 페이지 wishlist*2 reivew*1 cart*1 item*1

▼ 상품 상세 정보 요청

```
### 1) 상품 상세 페이지의 상품 정보 요청
GET /api/all/item/{pk}

auth: (없음)
body: (없음)
response:
- itemData: {
  itemPk: number,
  itemName: string,
  itemPrice: number,
  itemRegistDate: string (YYYY-MM-DD)
  imageUrl: string,      # 일단 C:\HUR\ornably-resource\item에 이미지 저장
  itemCategory: string,
  itemDiscountRate: number(0~100),
  itemDiscountPrice: number,
  itemAvgStar: number,
  itemWishlistToggle: boolean,  # 만약에 로그인 되어있는 상태라면 이거 찾아주기
}
errors:
- 404 NOT_FOUND
  - code: ITEM_NOT_FOUND | NEW_ITEMS_EMPTY
  - message: "요청에 맞는 상품이 없습니다"
- 400 BAD_REQUEST
  - code: INVALID_COUNT | COUNT_OUT_OF_RANGE
  - message: "요청 파라미터가 올바르지 않습니다"
- 500 SERVER
  - code: INTERNAL_SERVER_ERROR
  - message: "상품 정보를 불러오는 중 오류가 발생했습니다."
- 500 FILE_SYSTEM
  - code: IMAGE_RESOURCE_NOT_FOUND | IMAGE_PATH_INVALID
  - message: "상품 이미지 리소스를 찾을 수 없습니다."
- 500/timeout NETWORK
  - code: NETWORK_TIMEOUT
  - message: "잠시 후 다시 시도해주세요."
```

▶ 장바구니 담기 요청

▶ 리뷰 정보 요청

▶ 찜 상품 토글 끄기

▶ 찜 상품 토글 키기

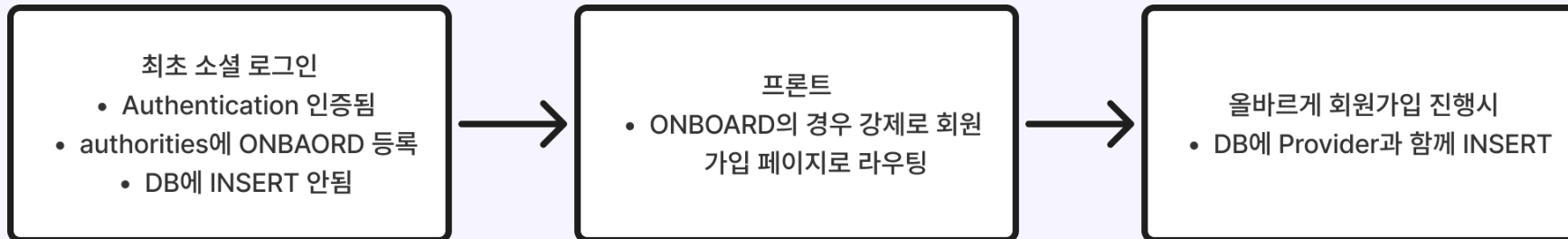
- 직관적인 API 명세서를 작성하여 개발 중 일관성과 작업 효율을 높임
- 이전 프로젝트 대비 병합 과정에서 발생하는 오류 빈도를 감소시킴
- 문제 발생 시 문서를 기준으로 원인을 빠르게 파악하여 불필요한 충돌을 최소화

로컬 사용자와 소셜 사용자의 병합 (1/2)

OAuth2 별 제공 데이터

OAuth2	식별 아이디	이메일	이름	주소	전화번호
Google	✓	✓	✓	✗	✗
Kakao	✓	□	□	✗	✗
Naver	✓	□	□	✗	✗

스프링 시큐리티를 통한 OAuth2의 회원가입 과정



로컬 사용자와 소셜 사용자의 병합 (2/2)

DefaultOAuth2UserService.loadUser 오버라이딩

```
24 @Service
25 public class CustomOAuth2UserService extends DefaultOAuth2UserService {
26
27     @Autowired //DB 접근을 담당하는 Repository Bean을 스프링으로부터 주입받음
28     private AccountRepository accountRepository;
29
30     @Override
31     public OAuth2User loadUser(OAuth2UserRequest userRequest) throws OAuth2AuthenticationException {
32
33
34         /* 1) 부모 클래스인 DefaultOAuth2UserService의 loadUser를 호출
35         OAuth 제공자(카카오, 구글) 서버에 사용자 정보 요청을 보내고 응답을 받아옴 userRequest = 로그인 요청 정보*/
36         OAuth2User oAuth2User = super.loadUser(userRequest);
37
38
39         // 2) 어떤 OAuth2제공자인지 확인 (카카오, 구글) - application.properties의 registrationId랑 동일
40         String registrationId = userRequest.getClientRegistration().getRegistrationId();
41
42         // 3) OAuth에서 내려준 사용자 정보 전체 (map형태)
43         Map<String, Object> attr = oAuth2User.getAttributes();
44
45         // 4) OAuth제공자별로 응답구조가 다르기때문에 공통형태로 정규화해서 꺼내기
46         SocialUserInfo socialUser = extractSocialUserInfo(registrationId, attr);
47
48         /* 5) 우리서비스에서 사용할 회원아이디 생성
49         provider prefix + providerId 예: kakao_123456789 / google_10987654321*/
50         String accountId = socialUser.provider() + "_" + socialUser.providerId();
51
52
53         // 6) DB에서 기존회원 여부 확인
54         //결과 있으면 기존회원 없으면 신규회원(온보딩 필요함)
55         AccountDTO accountDTO = new AccountDTO();
56         accountDTO.setCondition("SELECT_ACCOUNT_PK_BY_ACCOUNT_ID");
57         accountDTO.setAccountId(accountId);
58
59         AccountDTO result = accountRepository.selectOne(accountDTO);
60         Integer accountPk = (result != null) ? result.getAccountPk() : null ;
61
62
63
64
65         // 7) 기존/신규에 따라 권한 부여하기
66         Collection<? extends GrantedAuthority> authorities;
67         //회원Pk가 null이 아니면
68         if(accountPk != null) {
69             //기존회원
70             authorities = List.of(new SimpleGrantedAuthority("ROLE_USER"));
71
72         } else {
73             //신규회원 (온보딩 필요함)
74             authorities = List.of(new SimpleGrantedAuthority("ROLE_ONBOARD"));
75         }
76     }
77 }
```

로컬과 소셜 회원 모두 받을 수 있는 사용자 객체 다형성

```
26 public class OrnblyUser implements UserDetails, OAuth2User{
27     // 공유 멤버변수
28     private Integer accountPk;
29     private String accountName;
30     private String accountId;
31     private String accountRole;
32     private Collection<? extends GrantedAuthority> authorities;
33
34     // 로컬 회원 전용 멤버변수
35     private String accountPasswordHash;
36
37     // 소셜 회원 전용 멤버변수
38     private Map<String, Object> attributes;
39
40
41     // 1) 전체 필드 받는 생성자 (All-Args Constructor)
42     // - 모든 멤버변수를 한 번에 초기화할 때 사용
43     public OrnblyUser(Integer accountPk,
44                     String accountName,
45                     String accountId,
46                     String accountPasswordHash,
47                     String accountRole,
48                     Collection<? extends GrantedAuthority> authorities,
49                     Map<String, Object> attributes) {
50
51         // this.필드 = 파라미터; → 객체의 멤버변수에 값 세팅
52         this.accountPk = accountPk;
53         this.accountName = accountName;
54         this.accountId = accountId;
55         this.accountPasswordHash = accountPasswordHash;
56         this.accountRole = accountRole;
57         this.authorities = authorities;
58         this.attributes = attributes;
59     }
60 }
```

사용자의 권한 기반의 페이지 제한 계획

프론트의 권한별 허용 페이지 계획

	회원이 아닌 사용자	모든 사용자	회원	관리자	소셜 회원가입이 필요한 사용자
URL	NONE	EVERYONE	USER	ADMIN	ONBOARD
/, /items/**	✓	✓	✓	✓	✗
/login, /signup	✓	✗	✗	✗	✗
/onboard	✗	✗	✗	✗	✓
/account/**	✗	✗	✓	✗	✗
/admin/login	✓	✗	✗	✗	✗
/admin/** (/admin/login 제외)	✗	✗	✗	✓	✗
/403	✓	✓	✓	✓	✓

DB와 인증정보 기반의 권한의 정의

권한	조건
GUEST	authenticated = false DB에 저장되지 않는 개념 수준의 권한
USER	authenticated = true DB에 회원 존재
ADMIN	authenticated = true DB의 ACCOUNT_ROLE = 'ADMIN'
ONBOARD	authenticated = true DB에 회원 존재하지 않음

- 구현 전 미리 페이지에 필요한 권한을 확인
- DB에 존재하지 않는 인증된 회원의 인증객체의 권한을 ONBOARD로 설정하여 관리

AuthContext를 기반으로 한 페이지 권한 검사

< RouteProvider의 설정 >

```
46 export const router = createBrowserRouter([
47   { path: "/403", element: <Forbidden403 /> },
48
49   // PUBLIC (block onboard) + UserLayout
50   {
51     element: (
52       <Guard policy={ACCESS.PUBLIC_BLOCK_ONBOARD}>
53         <PublicLayout />
54       </Guard>
55     ),
56     errorElement: <RootError />,
57     children: [
58       { path: "/", element: <HomePage /> },
59       { path: "/items", element: <ItemListPage /> },
60       { path: "/items/:type", element: <ItemListPage /> },
61       { path: "/item/:itemPk", element: <ItemDetailPage /> },
62     ],
63   },
64
65   // USER ONLY + UserLayout
66   {
67     path: "/account",
68     element: (
69       <Guard policy={ACCESS.USER_ONLY}>
70         <PublicLayout />
71       </Guard>
72     ),
73     errorElement: <RootError />,
74     children: [
75       { index: true, element: <AccountPage /> },
76       { path: "address", element: <AddressListPage /> },
77       { path: "address/new", element: <AddressCreatePage /> },
78       { path: "review", element: <MyReviewsPage /> },
79       { path: "withdraw", element: <WithdrawPage /> },
80       { path: "cart", element: <CartPage /> },
81       { path: "order", element: <OrderListPage /> },
82       { path: "order/:orderPk", element: <OrderDetailPage /> },
```

- RouteProvider에 전달할 라우팅 생성 함수를 구현
- 부모 요소에 커스텀 Guard 컴포넌트를 적용해, ACCESS 정책 설정에 따라 페이지 접근/이동 흐름을 제어
- 접근이 허용된 페이지는 공통 레이아웃을 부모로 구성하여 화면 구조를 통일하고 컴포넌트 재사용성을 높임

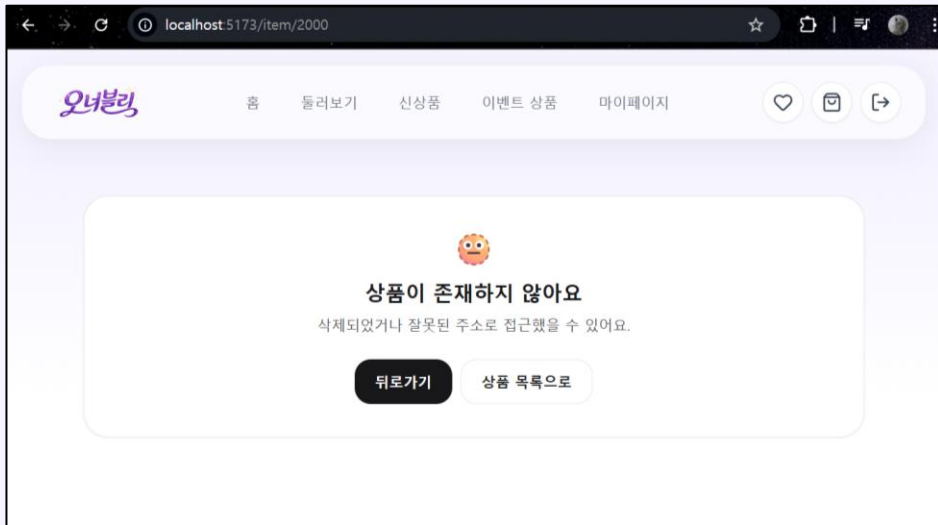
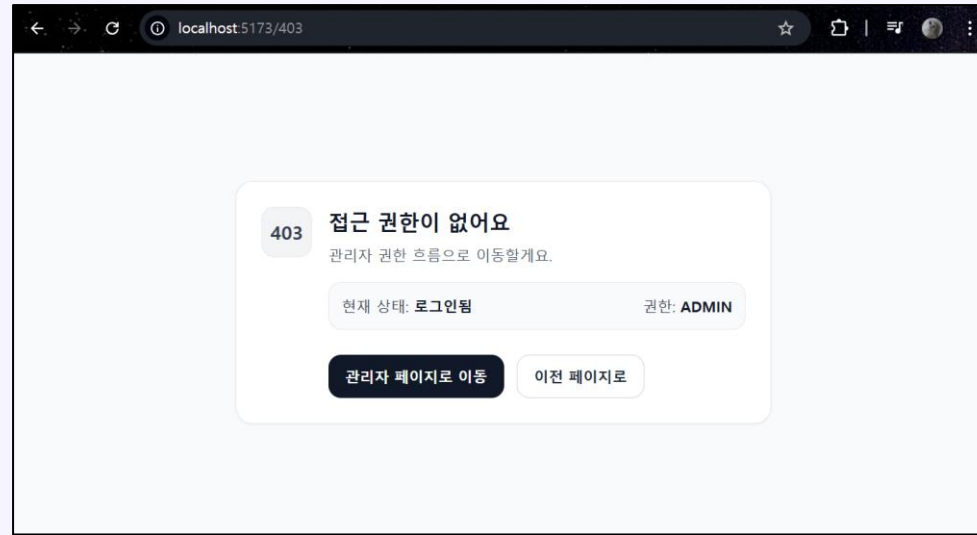
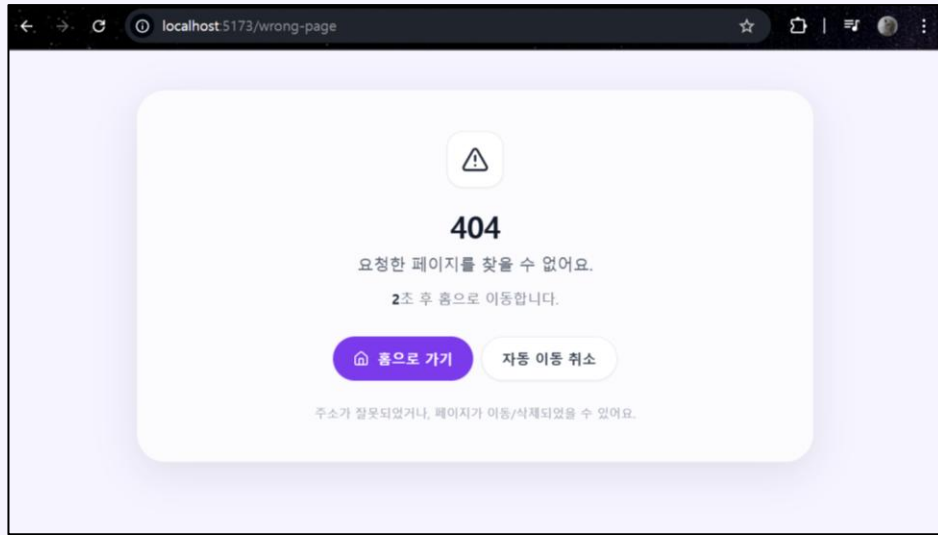
< Guard.jsx의 권한 판단 함수 >

```
25 function decideAccess({ status, isAuthenticated, authorities }, policy, locationPath) {
26   if (status === "loading") return { type: "loading" }; // 지금 상태 받아오는 중이면 기다려주기
27
28   const redirects = policy.redirects ?? {}; // policy의 금지 처리 경로 { unauthenticated | forbidden } 저장 또는 없이 저장
29   const unauthenticatedTo = redirects.unauthenticated ?? "/login"; // redirects.unauthenticated가 없으면 기본으로 로그인화면
30   const forbiddenTo = redirects.forbidden ?? "/403"; // redirects.forbidden이 없으면 기본으로 /403 화면
31   const authenticatedTo = redirects.authenticated ?? "/"; // 권한이 있어서 못하는 설정은 모두 홈페이지로 설정
32
33   // 1) guest-only
34   if (policy.mode === "guest") { // guest만 갈 수 있는 페이지 설정
35     if (isAuthenticated) return { type: "redirect", to: authenticatedTo }; // 로그인 되어있을 시 권한이 있어서 가야하는 페이지로 보내기
36     return { type: "allow" }; // 허용해주기
37   }
38
39   // 2) auth-required
40   if (policy.mode === "auth") { // 로그인되어있으며 권한 검사를 해야하는 경우
41     if (!isAuthenticated) { // 로그인이 안되어있을 때에는
42       return { type: "redirect", to: unauthenticatedTo, state: { from: locationPath } };
43     }
44     if (!hasAny(authorities, policy.allow)) { // 페이지에 대해 필요한 권한이 없으면 /403으로 내보내기
45       return { type: "redirect", to: forbiddenTo };
46     }
47     return { type: "allow" };
48   }
49
50   // 3) public (optional block)
51   if (policy.mode === "public") { // 모두가 가능하지만 일부를 막아야하는 경우
52     if (isAuthenticated && isBlocked(authorities, policy.block)) { // 인증은 되었지만 막혀있을때
53       return { type: "redirect", to: forbiddenTo }; // 403으로 보내기
54     }
55     return { type: "allow" };
56   }
57
58   // fallback
59   return { type: "allow" };
60 }
```

사용자 권한, 페이지 정책,
현재 경로를 기준으로
라우팅을 제어해

사용자 의도대로
페이지를 이동하도록 설계

사용자 흐름을 끊지 않는 예외 페이지 설계



잘못된 페이지 이동으로 인해 느낄 수 있는
사용자의 불편함을 최소화

활용한 다양한 기술 (1/2)

제한시간 동안 사용자 인증번호를 저장하기 위해 Redis 사용

```
40 public boolean sendCodeMessage(String targetPhone, String accountKey) {
41     // 전화번호 유효성 검사
42     if(!this.checkPhonePattern(targetPhone)) {
43         return false;
44     }
45
46     // 발송 메시지 작성하기
47     String code = this.generateCode(6);
48     String text = "오너블리 인증번호는 [" + code + "] 입니다.\r\n"
49         + "본인 확인을 위해 입력해 주세요.";
50
51     // SOLAPI 메시지 객체 생성
52     Message message = new Message();
53     message.setFrom(this.SOLAPI_API_NUMBER);
54     message.setTo(targetPhone);
55     message.setText(text);
56
57     // 만든 메시지 객체 보내기
58     MultipleDetailMessageSentResponse response;
59     try {
60         response = messageService.send(message, null);
61     } catch (SolapiMessageNotReceivedException e) {
62         e.printStackTrace();
63     } catch (SolapiEmptyResponseException e) {
64         e.printStackTrace();
65     } catch (SolapiUnknownException e) {
66         e.printStackTrace();
67     }
68
69     // 레디스 서버에 TTL 3분의 인증번호 저장하기
70     // getOtpKey를
71     String redisOtpKey = this.getOtpKey(accountKey);
72     this.redisTemplate.opsForValue()
73         .set(redisOtpKey, code, Duration.ofMinutes(3));
74
75     return true;
76 }
77 }
```

```
79 public boolean validateCode(String code, String key) {
80     // 키를 통해 code와 비교하여 일치하는지 반환하기
81     String redisOtpKey = this.getOtpKey(key);
82
83     // redis 서버로부터 OTP키를 통해 코드 가져오기
84     String redisCode = redisTemplate.opsForValue().get(redisOtpKey);
85
86     // 사용자의 입력과 DB의 코드 비교해서 반환
87     if(code.equals(redisCode)) {
88         return true;
89     }
90     else {
91         return false;
92     }
93 }
```

접속 사용자 수를 대시보드에 출력하기 위한 SessionListener

```
8 @Component
9 public class SessionCounter implements HttpSessionListener {
10
11     private static int count = 0;
12
13     @Override
14     public void sessionCreated(HttpSessionEvent se) {
15         count++;
16     }
17
18     @Override
19     public void sessionDestroyed(HttpSessionEvent se) {
20         count--;
21     }
22
23     public static int getCount() {
24         return count;
25     }
26 }
```

활용한 다양한 기술 (2/2)

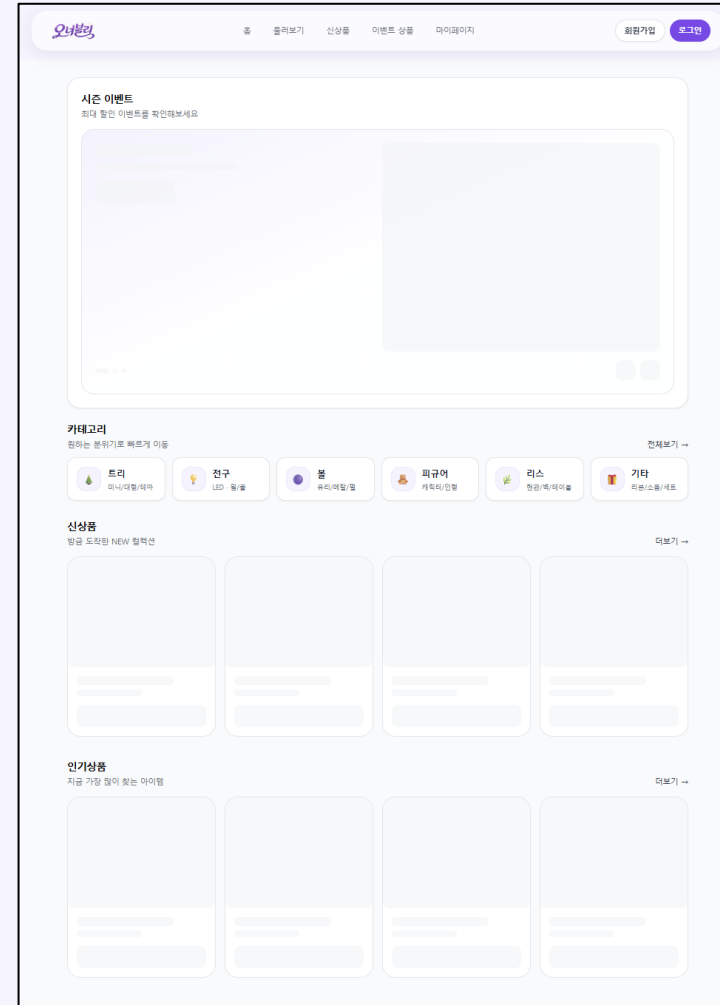
스프링의 리소스 핸들러 등록

```
8 @Configuration
9 public class WebConfig implements WebMvcConfigurer {
10
11     /* 현재 경로 상태
12     * resource.path=C:/HUR/workspace/Ornably/resource
13     * resource.path.review=C:/HUR/workspace/Ornably/resource/images/review
14     * resource.url-prefix=images
15     */
16
17     // application.properties에 적혀있는 resource폴더가 존재하는 경로
18     @Value("${resource.path}")
19     private String resourcePath; // C:/HUR/workspace/Ornably/resource
20     // resource.path에 접근하기 위한 prefix정보
21     @Value("${resource.url-prefix}")
22     private String urlPrefix; // /images
23
24     @Override
25     public void addResourceHandlers(ResourceHandlerRegistry registry) {
26         // 예: GET /images/review/abc.jpg -> file:C:/.../resource/images/review/abc.jpg
27         // 데이터를 줄 파일 경로 설정 (이후 .addResourceLocations() 를 통해 등록)
28         String location = "file:" + resourcePath + "/images/"; // C:/HUR/workspace/Ornably/resource/images/
29         registry.addResourceHandler(urlPrefix + "/*") // /images/* 요청을 리소스 요청으로 받아줌
30             .addResourceLocations(location);
31         // 스프링이 정적 리소스를 줄 때 http://localhost:8088/images/* 요청은 location으로부터 찾음
32     }
33 }
```

로깅을 위한 스프링의 AOP 활용

```
9 @Aspect
10 @Component
11 public class LogAdvice {
12     @Before("bugsandwich.ornably.common.PointcutCommon.controllerMethod()")
13     public void logMethodBefore(JoinPoint jp) {
14
15         System.out.println("[요청 받음 메서드] " + jp.getSignature().toString());
16         Object[] args = jp.getArgs();
17         for(Object arg:args) {
18             System.out.println(arg);
19         }
20         System.out.println("[요청 받음 메서드 끝]");
21     }
22
23     @AfterReturning(pointcut="bugsandwich.ornably.common.PointcutCommon.scontrollerMethod()", returning="result")
24     public void logMethodAfter(JoinPoint jp, Object result) {
25         System.out.println("[응답 발송 메서드] " + jp.getSignature().toString());
26         System.out.println("결과: [" + result + "]");
27         System.out.println("[응답 발송 메서드 끝]");
28     }
29 }
30 }
```

Skeleton UI를 활용



프로젝트 회고

- 팀 프로젝트를 통해 Git 협업, 설계 논의 등
개인 프로젝트에서는 경험하기 어려운 협업 과정을 직접 경험
 - 특히 신규 기술(React 등)을 도입해 팀 내 스터디로 이해도를 높이고
문서화를 통해 전체 흐름을 파악하는 방식으로 프로젝트 완성도를 높임
- Spring Boot 로 기존 프로젝트를 이관하고
결제 및 주문 처리 기능을 추가로 구현하며 서비스 구조를 깊이 이해
 - 이 과정에서 가독성이 좋은 코드의 중요성을 깨달았고
결제 과정에서 리팩토링을 통해 로직을 개선하며 성능 향상까지 경험
- 프로젝트에 몰입하여 완성도 있는 결과물을 구현할 수 있었고,
실제 서비스와 유사한 구조를 설계~구현해본 의미 있는 경험

버그샌드위치



◀ 노션 바로가기



◀ 백엔드 깃허브



◀ 프론트 깃허브

허 완



Email

hurwan0629@gmail.com

Tel

010-9294-1453

Blog

[태어났더니 개발이 너무 좋은 건에 대해여](#)

감사합니다

버그 샌드위치